

Introduction to C++ Programming, Input/Output & Operators

مقدمة في برمجة C++ والإدخال والإخراج والعمليات الحسابية

Dr. Doha Taha · Fall 2025

Faculty of Information Technology · MUST

Chapter 2 — C++ How to Program

1

First Program
أول برنامج

2

Modifying It
تعديل البرنامج

3

Adding Integers
جمع الأعداد

4

Memory
الذاكرة

5

Arithmetic
العمليات الحسابية

6

Comparisons
المقارنات

Q

Q&A + Output
أسئلة وتتبع

1

First Program: Printing a Line of Text

أول برنامج: طباعة سطر نصي

Fig. 2.1 — fig02_01.cpp

```
1 // Fig. 2.1: fig02_01.cpp
2 // Text-printing program.
3 #include <iostream> // enables program to output data to the screen
4
5 // function main begins program execution
6 int main() {
7     std::cout << "Welcome to C++!\n"; // display message
8
9     return 0; // indicate that program ended successfully
10 }
```

OUTPUT | المخرجات

Welcome to C++!

Read the program line by line

اقرأ البرنامج سطرًا بسطر

Line	What it does	ماذا يفعل
1-2	Comments: notes for humans.	تعليقات: ملاحظات للقارئ فقط.
3	<code>#include <iostream></code> adds the input/output tools.	<code>#include <iostream></code> يضيف أدوات الإدخال والإخراج.
6	<code>int main()</code> — the program starts here.	<code>int main()</code> — من هنا يبدأ تنفيذ البرنامج.
7	<code>std::cout << "..."</code> sends the text to the screen.	<code>std::cout << "..."</code> يرسل النص إلى الشاشة.
9	<code>return 0;</code> means "finished successfully".	<code>return 0;</code> يعني "انتهى البرنامج بنجاح".
10	<code>}</code> closes the body of <code>main</code> .	<code>}</code> يغلق جسم الدالة <code>main</code> .

Key ideas

الأفكار الأساسية

Term	Simple meaning	المعنى ببساطة
Comment <code>//</code>	A note for people. The compiler ignores it. <code>//</code> = one line. <code>/* ... */</code> = many lines.	ملاحظة للبشر والمترجم يتجاهلها. الرمز <code>//</code> لسطر واحد، والرمز <code>/* ... */</code> لعدة أسطر.

Preprocessor directive	A line that starts with <code>#</code> . It runs before compiling. It does not end with <code>;</code> .	سطر يبدأ بـ <code>#</code> ويُنفَّذ قبل الترجمة. ولا ينتهي بفاصلة منقوطة <code>;</code> .
<code><iostream></code>	Header file for input and output (<code>cout</code> , <code>cin</code>).	ملف الرأس الخاص بالإدخال والإخراج (<code>cout</code> و <code>cin</code>).
White space	Blank lines, spaces, tabs. The compiler ignores them. We use them to make code easy to read.	الأسطر الفارغة والمسافات والـ tabs. المترجم يتجاهلها، ونستخدمها لتنظيم الكود.
Function <code>main</code>	Every program has exactly one <code>main</code> . Execution always starts here.	كل برنامج فيه دالة <code>main</code> واحدة فقط ، ومنها يبدأ التنفيذ دائمًا.
<code>int</code> before <code>main</code>	<code>main</code> returns an integer (whole number) to the system.	الكلمة <code>int</code> تعني أن <code>main</code> ترجع عددًا صحيحًا.
Braces <code>{ }</code>	<code>{</code> starts the body of a function. <code>}</code> ends it.	القوس <code>{</code> يبدأ جسم الدالة والقوس <code>}</code> ينهيها.
Statement	An instruction that makes the computer do an action. Most end with <code>;</code> (the terminator).	أمر ينفذ إجراءً معينًا. ومعظمها ينتهي بـ <code>;</code> (منهي الجملة).
String literal	Text inside double quotes, like <code>"Welcome"</code> . Spaces inside a string are not ignored.	نص داخل علامتي تنصيص مثل <code>"Welcome"</code> ، والمسافات داخل النص لا تُهمل .
Stream	A flow of data. Input and output in C++ use streams.	تدفق من البيانات. الإدخال والإخراج في C++ يعتمدان على الـ streams.
<code>std::cout</code>	The standard output stream . It is connected to the screen.	مجرى الإخراج القياسي، ومتصل بالشاشة.
<code>std::cin / std::cerr</code>	<code>cin</code> = standard input (keyboard). <code>cerr</code> = standard error stream.	<code>cin</code> للإدخال من لوحة المفاتيح، و <code>cerr</code> لمجرى الأخطاء.
<code>std::</code>	<code>cout</code> , <code>cin</code> , <code>endl</code> belong to the namespace <code>std</code> , so we write <code>std::</code> before them.	الأسماء <code>cout</code> و <code>cin</code> و <code>endl</code> تتبع مجال الأسماء <code>std</code> لذلك نكتب <code>std::</code> قبلها.
<code><<</code> operator	Stream insertion operator . It puts the value on its right into the output stream.	عامل الإدراج : يضع القيمة التي على يمينه في مجرى الإخراج.
<code>>></code> operator	Stream extraction operator . It takes a value from the input stream.	عامل الاستخراج : يأخذ قيمة من مجرى الإدخال.
<code>return 0;</code>	<code>0</code> means the program ended successfully . If you forget it at the end of <code>main</code> , C++ assumes success.	الرقم <code>0</code> يعني أن البرنامج انتهى بنجاح ، وإذا نسيته في نهاية <code>main</code> يفترض C++ ذلك تلقائيًا.

Escape sequences

The backslash `\` is the **escape character**. With the next letter it makes an **escape sequence**: a special action or character. It is **not** printed as text.

تسلسلات الهروب

الشرطة المائلة العكسية `\` هي **حرف الهروب**، ومع الحرف التالي تكون **تسلسل هروب** ينفذ حركة خاصة أو يطبع رمزًا خاصًا، ولا تُطبع كنص.

Code	Meaning	المعنى
<code>\n</code>	Newline — cursor goes to the start of the next line.	سطر جديد — ينتقل المؤشر إلى بداية السطر التالي.
<code>\t</code>	Horizontal tab — move to the next tab stop.	مسافة tab أفقية — ينتقل المؤشر لموضع الـ tab التالي.
<code>\r</code>	Carriage return — back to the start of the current line.	العودة لبداية السطر الحالي دون النزول لسطر جديد.

<code>\a</code>	Alert — the system bell sounds.	تنبيه — يصدر صوت جرس النظام.
<code>\\</code>	Prints one backslash.	يطبع شرطة مائلة عكسية واحدة.
<code>\'</code>	Prints a single quote.	يطبع علامة تنصيص مفردة.
<code>\"</code>	Prints a double quote.	يطبع علامة تنصيص مزدوجة.

2 Modifying Our First Program

تعديل البرنامج الأول

A. One line, many statements. Two `cout` statements print on the **same line** because there is no `\n` between them.

أ. سطر واحد بعدة أوامر. أمر `cout` يطبعان على نفس السطر لأنه لا يوجد `\n` بينهما.

Fig. 2.3 — multiple statements

```
1 // Fig. 2.3: fig02_03.cpp
2 #include <iostream>
3
4 int main() {
5     std::cout << "Welcome ";
6     std::cout << "to C++!\n";
7 }
```

OUTPUT | المخرجات

Welcome to C++!

B. Many lines, one statement. Each `\n` moves to a new line. Two `\n` in a row give a **blank line**.

ب. عدة أسطر بأمر واحد. كل `\n` ينقل المؤشر لسطر جديد، وكتابة `\n` مرتين متتاليتين تعطي سطرًا فارغًا.

Fig. 2.4 — one statement, many lines

```
1 // Fig. 2.4: fig02_04.cpp
2 #include <iostream>
3
4 int main() {
5     std::cout << "Welcome\nto\n\nC++!\n";
6 }
```

OUTPUT | المخرجات

Welcome
to

C++!

3 Another Program: Adding Integers

برنامج آخر: جمع عددين صحيحين

The program asks the user for **two integers**, adds them, and prints the **sum**.

البرنامج يطلب من المستخدم **عددين صحيحين**، ثم يجمعهما ويطبع الناتج.

Fig. 2.5 — addition program

```

1 // Fig. 2.5: fig02_05.cpp
2 // Addition program that displays the sum of two integers.
3 #include <iostream> // enables program to perform input and output
4
5 // function main begins program execution
6 int main() {
7     // declaring and initializing variables
8     int number1{0}; // first integer to add (initialized to 0)
9     int number2{0}; // second integer to add (initialized to 0)
10    int sum{0}; // sum of number1 and number2 (initialized to 0)
11
12    std::cout << "Enter first integer: "; // prompt user for data
13    std::cin >> number1; // read first integer from user into number1
14
15    std::cout << "Enter second integer: "; // prompt user for data
16    std::cin >> number2; // read second integer from user into number2
17
18    sum = number1 + number2; // add the numbers; store result in sum
19
20    std::cout << "Sum is " << sum << std::endl; // display sum; end line
21 } // end function main

```

OUTPUT | المخرجات

```

Enter first integer: 45
Enter second integer: 72
Sum is 117

```

نصيحة · TIP

The numbers 45 and 72 are typed by the user.
Everything else is printed by the program.

الرقمان 45 و 72 يكتبهما المستخدم، وباقي النص يطبعه البرنامج.

Variables & declarations**المتغيرات والتعريفات**

Term	Simple meaning	المعنى ببساطة
Declaration	Introduces a name (identifier) into the program.	يُعرّف اسماً (identifier) جديداً داخل البرنامج.
Variable	A location in memory where a value is stored for the program to use.	مكان في الذاكرة تُخزّن فيه قيمة يستخدمها البرنامج.
Rule	Every variable must have a name and a data type before you use it.	كل متغير يجب أن يكون له اسم ونوع بيانات قبل استخدامه.
List initialization	<code>int sum{0};</code> gives the variable a starting value. Added in C++11. Old style: <code>int sum = 0;</code>	الصيغة <code>int sum{0};</code> تعطي المتغير قيمة ابتدائية (من C++11). والصيغة القديمة: <code>int sum = 0;</code>
Comma-separated list	Declare several names together: <code>int a, b, c;</code>	تعريف عدة أسماء معاً مفصولة بفواصل: <code>int a, b, c;</code>
Where to declare?	Almost anywhere, but before the first use.	تقريباً في أي مكان، لكن قبل أول استخدام.

Type	Stores	يخزن
<code>int</code>	Whole numbers: 7, -11, 0, 31914	أعدادًا صحيحة: 7 و -11 و 0 و 31914
<code>double</code>	Real numbers (with a decimal point): 2.5, 0.0, -13.5	أعدادًا حقيقية (بها كسر عشري): 2.5 و 0.0 و -13.5
<code>char</code>	One character only: a letter, a digit, or a symbol like \$ or *	حرفًا واحدًا فقط: حرف أو رقم أو رمز مثل \$ أو *

REMEMBER · احفظ

Type names are keywords, so they must be written in **lowercase**: `int`, not `Int`.

أسماء الأنواع كلمات محجوزة (keywords) ويجب كتابتها بحروف صغيرة: `int` وليس `Int`.

Valid names (identifiers)

أسماء المتغيرات الصحيحة

- An identifier uses **letters, digits and underscores** only. يتكوّن الاسم من حروف وأرقام وشرطة سفلية فقط.
- It must **not start with a digit**. لا يجوز أن يبدأ برقم.
- It must **not be a keyword** (like `int`). لا يجوز أن يكون كلمة محجوزة مثل `int`.
- C++ is **case sensitive**: `a1` and `A1` are **different**. لغة C++ حساسة لحالة الأحرف: فالاسمان `a1` و `A1` مختلفان.

Input & output tools

أدوات الإدخال والإخراج

Term	Simple meaning	المعنى ببساطة
Prompt	A message that tells the user what to do, e.g. <code>"Enter first integer: "</code> .	رسالة توجّه المستخدم لما يفعله، مثل <code>"Enter first integer: "</code> .
<code>cin >> x;</code>	Waits for the user, reads a value from the keyboard , and stores it in <code>x</code> .	ينتظر المستخدم ويقرأ قيمة من لوحة المفاتيح ويخزنها في <code>x</code> .
How input works	The user types the number (as characters) and presses Enter . C++ converts it to an integer and copies it into the variable.	يكتب المستخدم الرقم (كحروف) ثم يضغط Enter ؛ فيحوّله C++ إلى عدد صحيح وينسخه في المتغير.
Assignment <code>=</code>	<code>sum = number1 + number2;</code> calculates the right side, then stores the result in the left variable.	الأمر <code>sum = number1 + number2;</code> يحسب الطرف الأيمن ثم يخزن الناتج في المتغير على اليسار.
Binary operator	An operator with two operands , like <code>+</code> and <code>=</code> .	عامل له معاملان مثل <code>+</code> و <code>=</code> .
<code>std::endl</code>	A stream manipulator . It prints a newline and flushes the output buffer (forces waiting output to show on screen now).	أداة تحكم في المجرى: تطبع سطرًا جديدًا وتفرّغ الذاكرة المؤقتة للإخراج (تُظهر ما هو منتظر على الشاشة فورًا).
Chaining <code><<</code>	Using many <code><<</code> in one statement: <code>cout << "Sum is " << sum << endl;</code> . Also called concatenating or cascading .	استخدام عدة <code><<</code> في أمر واحد، ويسمى سلسلة أو تتابعًا (chaining / cascading).
Calculations in output	You can calculate inside <code>cout</code> : <code>cout << a + b;</code>	يمكن إجراء الحساب داخل أمر <code>cout</code> مباشرة: <code>cout << a + b;</code>

REMEMBER · احفظ

`\n` vs `std::endl` : both start a new line, but `endl` also **flushes** the buffer.

الفرق بين `\n` و `std::endl` : كلاهما ينقل لسطر جديد، لكن `endl` يفرغ الذاكرة المؤقتة أيضًا.

4 Memory Concepts

مفاهيم الذاكرة

- Variable names (`number1` , `number2` , `sum`) match **locations in memory**.
- Every variable has a **name, type, size and value**.
- Putting a new value in a location **overwrites** the old one. This is **destructive**.
- Reading a value out of a location is **non-destructive**: the value stays.

- أسماء المتغيرات (`number1` , `number2` , `sum`) تقابل أماكن في الذاكرة.
- لكل متغير اسم ونوع وحجم وقيمة.
- وضع قيمة جديدة في المكان **يحوط** القيمة القديمة، وهذا يسمى عملية **مدمرة** (destructive).
- قراءة القيمة من المكان **غير مدمرة** (non-destructive): تبقى القيمة كما هي.

After `cin >> number1` (45)

بعد إدخال الرقم الأول

number1 45

After `cin >> number2` (72)

بعد إدخال الرقم الثاني

number1 45
number2 72

After `sum = number1 + number2`

بعد حساب المجموع

number1 45
number2 72
sum 117

نصيحة · TIP

Adding does not change `number1` or `number2` (reading is non-destructive). Only `sum` receives a new value.

عملية الجمع لا تغيّر `number1` ولا `number2` (القراءة غير مدمرة)، فقط `sum` تستلم قيمة جديدة.

5 Arithmetic in C++

العمليات الحسابية في C++

Operation	Operator	Algebra	C++	العملية
Addition	+	$f + 7$	<code>f + 7</code>	الجمع
Subtraction	-	$p - c$	<code>p - c</code>	الطرح
Multiplication	*	$b \cdot m$	<code>b * m</code>	الضرب
Division	/	$x \div y$	<code>x / y</code>	القسمة
Remainder	%	$r \bmod s$	<code>r % s</code>	باقي القسمة

- All arithmetic operators are **binary** (two operands).
- **%** gives the **remainder** after integer division. It works **only with integers**. Example: $7 \% 2 = 1$.
- **Integer division** (int ÷ int) gives an int. The fraction is **cut off (truncated)**, not rounded. Example: $7 / 2 = 3$.
- Write expressions in **straight-line form**: "a divided by b" is a / b .
- Use parentheses like in algebra: $a * (b + c)$.
- There is **no power operator**. Write $x * x$ for x^2 .
- كل العمليات الحسابية ثنائية (لها معاملان).
- العامل **%** يعطي باقي القسمة ويعمل مع الأعداد الصحيحة فقط. مثال: $7 \% 2 = 1$.
- **القسمة الصحيحة** (int ÷ int) تعطي عددًا صحيحًا، ويُحذف الكسر (truncation) دون تقريب. مثال: $7 / 2 = 3$.
- نكتب التعبير في **سطر مستقيم**: "a مقسومة على b" تُكتب a / b .
- نستخدم الأقواس كما في الجبر: $a * (b + c)$.
- **لا يوجد** عامل للأس، فنكتب $x * x$ بدل x^2 .

Operator precedence

أولوية العمليات

Order	Operators	Rule	القاعدة
1st	()	Evaluated first. For nested parentheses, the innermost pair goes first.	الأقواس أولًا، وفي الأقواس المتداخلة يُحسب الأعمق أولًا.
2nd	* / %	Evaluated second. Several of them: left to right .	ثانيًا، وإذا تعددت تُنقذ من اليسار لليمين .
3rd	+ -	Evaluated last. Several of them: left to right .	أخيرًا، وإذا تعددت تُنقذ من اليسار لليمين .

WATCH OUT · انتبه

For $(a + b) * (c - d)$ — two separate groups at the same level — the C++ standard does **not** say which group is calculated first.

في التعبير $(a + b) * (c - d)$ — مجموعتان في نفس المستوى — لا يحدد معيار ++C أي مجموعة تُحسب أولًا.

Example: second-degree polynomial

مثال: كثيرة حدود من الدرجة الثانية

Calculate $y = 2 * 5 * 5 + 3 * 5 + 7$; (that is $y = ax^2 + bx + c$ with $a=2, b=3, c=7, x=5$). احسب $y = 2 * 5 * 5 + 3 * 5 + 7$; (أي $y = ax^2 + bx + c$ حيث $a=2$ و $b=3$ و $c=7$ و $x=5$).

Step	Expression	What happens	ماذا يحدث
1	$2 * 5 * 5 + 3 * 5 + 7$	$2 * 5 = 10$ (leftmost multiplication)	أول ضرب من اليسار
2	$10 * 5 + 3 * 5 + 7$	$10 * 5 = 50$	الضرب التالي
3	$50 + 3 * 5 + 7$	$3 * 5 = 15$ (multiply before add)	الضرب قبل الجمع
4	$50 + 15 + 7$	$50 + 15 = 65$ (leftmost addition)	أول جمع من اليسار
5	$65 + 7$	$65 + 7 = 72$	آخر عملية جمع
6	$y = 72$	Assignment is last: 72 is stored in y	الإسناد آخرًا: تُخزن 72 في y

TIP · نصيحة

Extra (redundant) parentheses are allowed. They make the expression clearer:

$y = (2 * 5 * 5) + (3 * 5) + 7$;

يمكن استخدام أقواس زائدة لتوضيح التعبير، مثل:

$y = (2 * 5 * 5) + (3 * 5) + 7$;

6 Equality & Relational Operators

عوامل المساواة والمقارنة

Needed for the output questions at the end. A comparison gives **1 (true)** or **0 (false)**. The `if` statement runs its action only when the condition is true.

ضرورية لأسئلة المخرجات في النهاية. المقارنة تعطي **1 (صحيح)** أو **0 (خطأ)**. وأمر `if` ينقذ ما بعده فقط إذا كان الشرط صحيحًا.

Operator	Meaning	Example (a=2, b=2)	المعنى
<code>==</code>	equal to	<code>a == b</code> → 1	يساوي
<code>!=</code>	not equal to	<code>a != b</code> → 0	لا يساوي
<code>></code>	greater than	<code>a > b</code> → 0	أكبر من
<code><</code>	less than	<code>a < b</code> → 0	أصغر من
<code>>=</code>	greater than or equal	<code>a >= b</code> → 1	أكبر من أو يساوي
<code><=</code>	less than or equal	<code>a <= b</code> → 1	أصغر من أو يساوي

WATCH OUT · انتبه

`=` **stores** a value (assignment). `==` **compares** two values. Do not mix them!

الرمز `=` **يخزن** قيمة (إسناد)، أما `==` **ف يقارن** قيمتين. لا تخط بينهما!

REMEMBER · احفظ

Without braces, `if` controls **only the next statement**. With `{ }`, it controls everything inside the braces.

بدون أقواس `{ }` يتحكم `if` في **الأمر التالي فقط**، ومع الأقواس يتحكم في كل ما بداخلها.

Medium level

المستوى المتوسط

1 What is the purpose of the `#include <iostream>` directive? ما الغرض من التوجيه `#include <iostream>` ؟

Answer It adds the header file that gives us input/output tools such as `cout` and `cin`.

الإجابة يضيف ملف الرأس الذي يوفر أدوات الإدخال والإخراج مثل `cout` و `cin`.

2 Why are comments important in a C++ program? لماذا تُعدّ التعليقات مهمة في برنامج C++ ؟

Answer They explain the code, so it is easier to read and maintain. The compiler ignores them, so they do not change how the program runs.

الإجابة لأنها تشرح الكود فيسهل فهمه وصيانته، والمترجم يتجاهلها فلا تؤثر على تنفيذ البرنامج.

3 What is the role of the `main()` function? ما دور الدالة `main()` ؟

Answer It is the **entry point**: the program starts running here. Every C++ program must have exactly one `main()`.

الإجابة هي نقطة البداية: ينطلق منها تنفيذ البرنامج، ويجب أن يحتوي كل برنامج على `main()` واحدة فقط.

4 How can you output a string in C++? كيف تطبع نصًا (string) في C++ ؟

Answer Use `std::cout` with the `<<` operator, for example `std::cout << "Hello";`.

الإجابة نستخدم `std::cout` مع العامل `<<` مثل: `std::cout << "Hello";`.

5 Explain the use of escape sequences like `\n`. اشرح استخدام تسلسلات الهروب مثل `\n`.

Answer They control the format of the output. `\n` starts a new line, `\t` adds a tab, and `\\` prints a backslash.

الإجابة تتحكم في شكل المخرجات: `\n` سطر جديد، و `\t` مسافة tab، و `\\` تطبع شرطة مائلة عكسية.

Hard level

المستوى الصعب

1 Why is variable initialization important? لماذا تُعدّ تهيئة المتغيرات مهمة؟

Answer A variable that is not initialized holds an unknown (garbage) value. This can make the program behave in unpredictable ways. Example: `int sum{0};`

الإجابة المتغير غير المهيأ يحمل قيمة عشوائية مجهولة، فيؤدي ذلك إلى سلوك غير متوقع للبرنامج. مثال: `int sum{0};`

2 What is a destructive operation in memory? ما العملية المدمرة (destructive) في الذاكرة؟

Answer Storing a new value in a memory location **overwrites** the old value, so the old data is lost. Reading a value is non-destructive.

الإجابة هي تخزين قيمة جديدة في مكان بالذاكرة فتُحذف القيمة القديمة. أما قراءة القيمة فهي غير مدمرة.

3 Explain the precedence rules for C++ arithmetic operators.

Answer 1) Parentheses `( )` first. 2) Then `*`, `/`, `%` (left to right). 3) Then `+`, `-` (left to right). Parentheses can change the order.

اشرح قواعد أولوية العمليات الحسابية في C++.

الإجابة (1) الأقواس `( )` أولاً. (2) ثم `*` و `/` و `%` (من اليسار لليمين). (3) ثم `+` و `-` (من اليسار لليمين). والأقواس تغيّر الترتيب.

4 How does `std::endl` differ from `\n`?

Answer Both make a new line. But `std::endl` also flushes the output buffer, so the text shows on the screen immediately.

ما الفرق بين `std::endl` و `\n` ؟

الإجابة كلاهما ينشئ سطرًا جديدًا، لكن `std::endl` يفرّغ الذاكرة المؤقتة للإخراج أيضًا فيظهر النص فورًا على الشاشة.

5 What are fundamental types in C++?

Answer Built-in types such as `int` (whole numbers), `double` (real numbers) and `char` (one character). Their names are lowercase keywords.

ما الأنواع الأساسية (fundamental types) في C++؟

الإجابة أنواع مدمجة في اللغة مثل `int` (أعداد صحيحة) و `double` (أعداد حقيقية) و `char` (حرف واحد)، وأسمائها كلمات مدمجة بحروف صغيرة.

نصيحة · TIP

These slides use an old compiler style (`main()` without `int`, `<conio>`, `getch()`). In modern C++ write `int main()` . Here we only care about the logic. In all questions `c = a == b` means: compare first, then store 1 or 0 in `c` .

هذه الشرائح بأسلوب مترجم قديم (`main()` بدون `int` والمكتبة `<conio>` والدالة `getch()`). في C++ الحديثة نكتب `int main()` . المهم هنا هو المنطق فقط. وفي كل الأسئلة `c = a == b` تعني: قارن أولاً ثم خزن 1 أو 0 في `c` .

Comparison operators

عوامل المقارنة

1

```
int a, b, c;
a = 1;
b = 2;
c = a == b;
cout << c;
```

المخرجات | ANSWER

0

Why `1 == 2` is false, so `c` becomes 0 (F).

السبب المقارنة `1 == 2` خاطئة لذلك تصبح قيمة `c` تساوي 0 (F).

2

```
int a, b, c;
a = 2;
b = 2;
c = a == b;
cout << c;
```

المخرجات | ANSWER

1

Why `2 == 2` is true, so `c` becomes 1 (T).

السبب المقارنة `2 == 2` صحيحة لذلك تصبح `c` تساوي 1 (T).

3

```
int a, b, c;
a = 2;
b = 2;
c = a != b;
cout << c;
```

المخرجات | ANSWER

0

Why `2 != 2` is false (they are equal), so the result is 0.

السبب المقارنة `2 != 2` خاطئة (القيمتان متساويتان) لذلك الناتج 0.

4

```
int a, b, c;
a = 1;
b = 2;
c = a < b;
cout << c;
```

المخرجات | ANSWER

1

Why `1 < 2` is true, so the result is 1.

السبب المقارنة `1 < 2` صحيحة لذلك الناتج 1.

5

```
int a, b, c;
a = 2;
b = 2;
c = a <= b;
cout << c;
```

المخرجات | ANSWER

1

Why `2 <= 2` is true (equal counts), so the result is 1.

السبب المقارنة `2 <= 2` صحيحة (المساواة مقبولة) لذلك الناتج 1.

5b Slide extra: in Question 5 the slide also asks you to **test** the other two operators with $a = 2$, $b = 2$.

إضافة من الشريحة: في السؤال 5 تطلب الشريحة أيضًا أن تختبر العاملين الآخرين بالقيم $a = 2$ و $b = 2$.

Test	Result	السبب
<code>c = a > b;</code>	0	2 ليست أكبر من 2 → خطأ
<code>c = a >= b;</code>	1	2 أكبر من أو تساوي 2 → صحيح

The if statement

أمر if

6

```
int a, b;
a = 2;
b = 2;

if (a==b)
    cout << endl << "1";
cout << endl << "2";
cout << endl << "3";
```

المخرجات | ANSWER

1
2
3

Why The condition is true, so line "1" runs. Lines "2" and "3" always run. Each `endl` starts a new line (the first line is empty).

السبب الشرط صحيح فتنفذ أمر طباعة "1"، أما "2" و "3" فتُنفَّذان دائمًا. وكل `endl` يبدأ سطرًا جديدًا (والسطر الأول فارغ).

7

```
int a, b;
a = 1;
b = 2;

if (a==b)
    cout << endl << "1";
cout << endl << "2";
cout << endl << "3";
```

المخرجات | ANSWER

2
3

Why The condition is false, so only the **next** statement ("2") is skipped. "2" and "3" still run.

السبب الشرط خاطئ فيتجاهل الأمر التالي فقط (طباعة "1")، وتبقى "2" و "3" تُنفَّذان.

8

```
int a, b;
a = 1;
b = 2;

if (a==b)
{
    cout << endl << "1";
    cout << endl << "2";
}
cout << endl << "3";
```

المخرجات | ANSWER

3

Why The condition is false. The braces group "1" and "2" together, so both are skipped. Only "3" runs.

السبب الشرط خاطئ، والأقواس تجمع "1" و "2" معًا، فيتجاهلان معًا، ولا يُنفَّذ إلا "3".

1 What is $7 / 2$ when both numbers are `int` ?

كم تساوي $7 / 2$ إذا كان العددين من نوع `int` ؟

Answer 3. Integer division cuts off the fraction (no rounding).

الإجابة 3. القسمة الصحيحة تحذف الكسر دون تقريب.

2 What is $17 \% 5$?

كم تساوي $17 \% 5$ ؟

Answer 2. $17 = 3 \times 5 + 2$, and the remainder is 2.

الإجابة 2. لأن $17 = 3 \times 5 + 2$ والباقي 2.

3 Calculate $2 + 3 * 4$ and $(2 + 3) * 4$.

احسب $2 + 3 * 4$ و $(2 + 3) * 4$.

Answer 14 and 20. Multiplication goes first; parentheses change the order.

الإجابة 14 و 20. الضرب أولاً، والأقواس تغيّر الترتيب.

4 Calculate $20 / 5 * 2$.

احسب $20 / 5 * 2$.

Answer 8. Same level, so left to right: $20 / 5 = 4$, then $4 * 2 = 8$.

الإجابة 8. نفس المستوى فنحسب من اليسار لليمين: $20 / 5 = 4$ ثم $4 * 2 = 8$.

5 Which names are valid: `total_1`, `2nd`, `int`, `Sum` ?

أي هذه الأسماء صحيحة: `total_1` و `2nd` و `int` و `Sum` ؟

Answer `total_1` and `Sum` are valid. `2nd` starts with a digit. `int` is a keyword.

الإجابة الصحيح: `total_1` و `Sum`. والاسم `2nd` يبدأ برقم، و `int` كلمة محجوزة.

6 Are `count` and `Count` the same variable?

هل `count` و `Count` نفس المتغير؟

Answer No. C++ is case sensitive, so they are two different identifiers.

الإجابة لا. لغة C++ حساسة لحالة الأحرف فهما اسمان مختلفان.

7 What is the difference between `=` and `==` ?

ما الفرق بين `=` و `==` ؟

Answer `=` stores a value in a variable. `==` compares two values and gives 1 (true) or 0 (false).

الإجابة الرمز `=` يخزن قيمة في متغير، و `==` يقارن بين قيمتين ويعطي 1 (صحيح) أو 0 (خطأ).

8 What do `<<` and `>>` do?

ماذا يفعل `<<` و `>>` ؟

Answer `<<` sends data out (with `cout`). `>>` takes data in (with `cin`).

الإجابة العامل `<<` يرسل البيانات للخارج (مع `cout`)، والعامل `>>` يأخذ البيانات للداخل (مع `cin`).

9 Why do we write `std::` before `cout` ?

لماذا نكتب `std::` قبل `cout` ؟

Answer Because `cout` belongs to the namespace `std`. The prefix tells C++ where to find the name.

الإجابة لأن `cout` يتبع مجال الأسماء `std` ، والبادئة تخبر ++C أين تجد الاسم.

10 What does `cout << "Hi\n\n";` print?

ماذا يطبع الأمر `cout << "Hi\n\n";` ؟

Answer The word `Hi`, then the cursor goes down two lines, which leaves **one blank line**.

الإجابة يطبع الكلمة `Hi` ثم ينزل المؤشر سطرين فيظهر سطر فارغ بعدها.

Quick Review Sheet

ورقة مراجعة سريعة

Program skeleton · هيكل البرنامج

```
#include <iostream>

int main() {
    // statements here;
    return 0;
}
```

Common mistakes · أخطاء شائعة

- Missing `;` after a statement · نسيان `;`
- `;` after `#include` · وضع `;` بعد `#include`
- Forgetting `std::` · نسيان `std::`
- Using `=` instead of `==` · استخدام `=` بدل `==`
- Using a variable before declaring it · استخدام متغير قبل تعريفه
- `Int` instead of `int` · كتابة `Int` بدل `int`

Remember · احفظ

- `cout << out` · `cin >> in`
- Reading = non-destructive · القراءة غير مدققة
- Writing = destructive · الكتابة مدققة
- `7 / 2 = 3` and `7 % 2 = 1`
- Order: `( )` → `* / %` → `+ -`

Comparisons give · المقارنة تعطي

- True = 1 · صحيح
- False = 0 · خطأ
- `if` without `{ }` = next line only · بدون أقواس السطر التالي فقط